

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №5

Реализация межсервисного взаимодействия посредством
очередей сообщений

Выполнил:

Христофоров Владислав

Группа

К3340

Поток

БР 1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подключить и настроить брокер сообщений RabbitMQ (или Kafka). Реализовать межсервисное взаимодействие с использованием механизмов очередей сообщений для перехода от синхронной связи к событийно-ориентированной архитектуре (Event-Driven Architecture).

Ход работы

1 Интеграция RabbitMQ в инфраструктуру

В состав инфраструктуры проекта (файл `docker-compose.yml`) добавлен контейнер брокера сообщений `rabbitmq:3.8`. В целевые Node.js сервисы установлена библиотека-клиент `amqpplib`.

2 Архитектурный паттерн Издатель-Подписчик

В качестве бизнес-сценария для асинхронного взаимодействия выбрана Оркестрация каскадного удаления данных. В предыдущей лабораторной работе при блокировке (бане) пользователя Identity Service делал синхронные вызовы (Axios) к другим сервисам. Этот подход заменен на использование RabbitMQ:

- разработан утилитный класс `RabbitMQService`, реализующий логику подключения (с механизмом `Retry Policy`), публикации (`publish`) и подписки (`consume`),
- в брокере создан обменник (`Exchange`) типа `fanout` с именем `user_events`. Тип `fanout` выбран осознанно, так как событие блокировки пользователя должно быть разослано всем заинтересованным микросервисам без сложной маршрутизации.

3 Реализация Издателя

Контроллер администратора в Identity Service был модифицирован. Теперь при вызове `DELETE /admin/users/{id}/ban` сервис не блокирует поток исполнения в ожидании ответа от других сервисов. Вместо этого он выполняет локальное мягкое удаление профиля и публикует сообщение: `rabbitMQService.publish('user_events', 'user.deleted', { userId: id });`

4 Реализация Подписчиков и Acknowledgement

На этапе инициализации приложений Recipe Service и Social Service устанавливается соединение с RabbitMQ, декларируются свои собственные очереди и привязываются их к обменнику user_events.

При получении сообщения сервисы выполняют независимую каскадную очистку (удаление рецептов, блогов, комментариев и лайков). После успешной обработки данных в БД вызывается метод `channel.ack(msg)` – явное подтверждение успешной обработки (Acknowledgement). Если сервис упадет в процессе очистки БД, сообщение останется в очереди и будет обработано после рестарта.

Вывод

В ходе выполнения домашнего задания успешно осуществлен переход от жестко связанной синхронной оркестрации к событийно-ориентированной архитектуре. Использование брокера RabbitMQ повысило отказоустойчивость системы: сервисы теперь не зависят от сетевой доступности друг друга в момент удаления пользователя, а применение механизма Acknowledgement гарантирует 100% доставку и обработку каскадных удалений.